

Chapter 3

邏輯閘層次的最小化

二進位元邏輯

圖示法

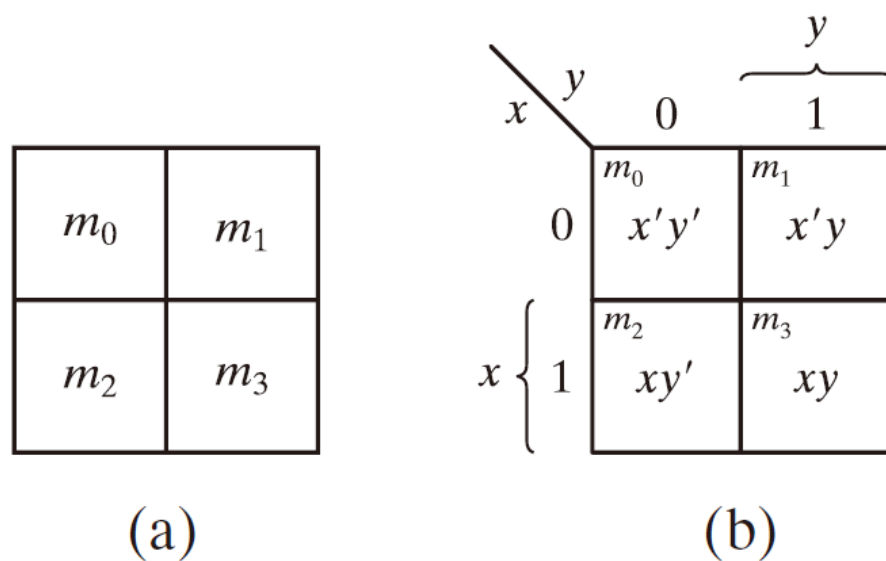
- ▶ 因為布林代數沒有一套固定的步驟可以保證你下一步一定怎麼做。很多時候我們只能靠經驗去嘗試化簡，所以過程常常會變得很繁瑣。
- ▶ 因此在數位邏輯設計中，工程師常使用另一種方法來化簡布林函數，就是 **卡諾圖 (Karnaugh map)**，也稱為 **K-map**。
- ▶ 卡諾圖有幾個重要特點：
 - ▶ 它是一種比較簡單、直觀的化簡方法
 - ▶ 可以把它看成是**真值表**的圖解
- ▶ 也就是說，我們把真值表中的每一種輸入組合，用圖上的一個方格來表示。
- ▶ 整個卡諾圖是由許多正方形格子所組成，而且每一個方格都代表一個 **minterm**，也就是一個**全及項**。

圖示法

- ▶ 當我們利用卡諾圖完成化簡之後，最後得到的布林函數表示式，通常會是兩種標準形式中的其中一種：
- ▶ **積項和 (Sum of Products, SOP)**
- ▶ **和項積 (Product of Sums, POS)**
- ▶ 以最簡的代數表示式就是該代數表示式的項數最少
- ▶ 每一項的文字變數的數目最少
- ▶ 這種表示式所產生的電路圖，其閘數最少且閘的輸入數最少
- ▶ 最簡表示式並不是唯一的

二變數卡諾圖

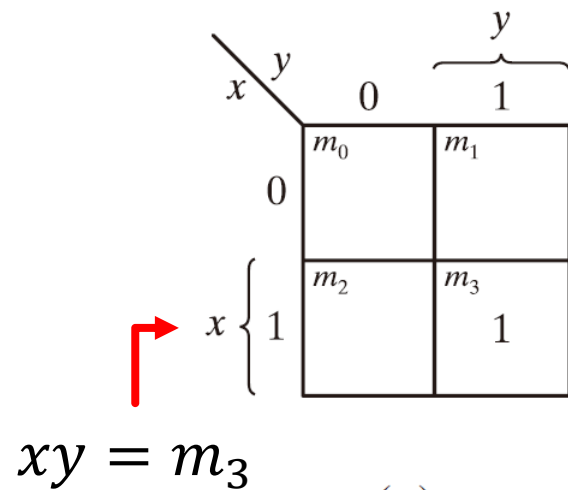
- ▶ 二變數之卡諾圖有四個全及項



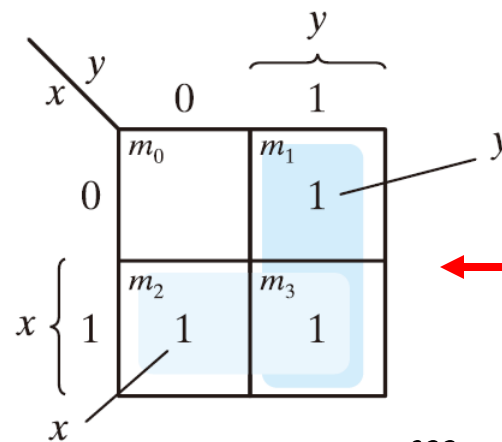
▶ 圖 3.1 二變數卡諾圖

二變數卡諾圖

- 如果我們能標記圖中的方格為一已知函數的全及項，則此二變數的卡諾圖，就可代表二變數所形成的16個布林函數中的任一個 (每一個格子有1或是0兩種選擇)



(a) xy



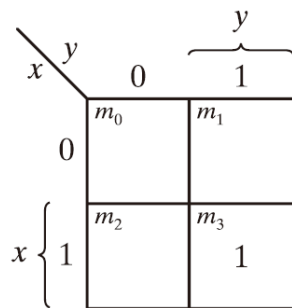
(b) $x + y$

$$m_1 + m_2 + m_3 = x'y + xy' + xy = x + y$$

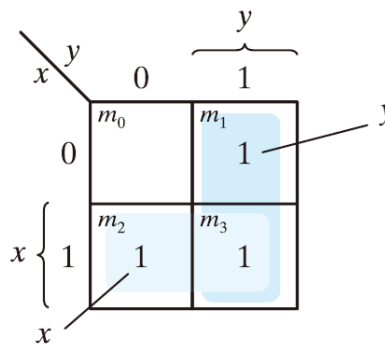
圖 3.2 函數在卡諾圖中的表示法

二變數卡諾圖

- ▶ 假設我們有兩個變數 x 和 y ，那麼所有可能的輸入組合有4種情況 (00,01,10,11)。
- ▶ 因此 二變數卡諾圖就會有 4 個方格，每一個方格代表一個 **minterm** (全及項)。
- ▶ 在圖中： $m_0 = x'y'$ ， $m_1 = x'y$ ， $m_2 = xy'$ ， $m_3 = xy$ ，所以每一個格子其實就是一個 **minterm** 的位置。
- ▶ 如果某一組輸入時函數輸出是 1，我們就把 1 填在對應的格子裡。



(a) xy

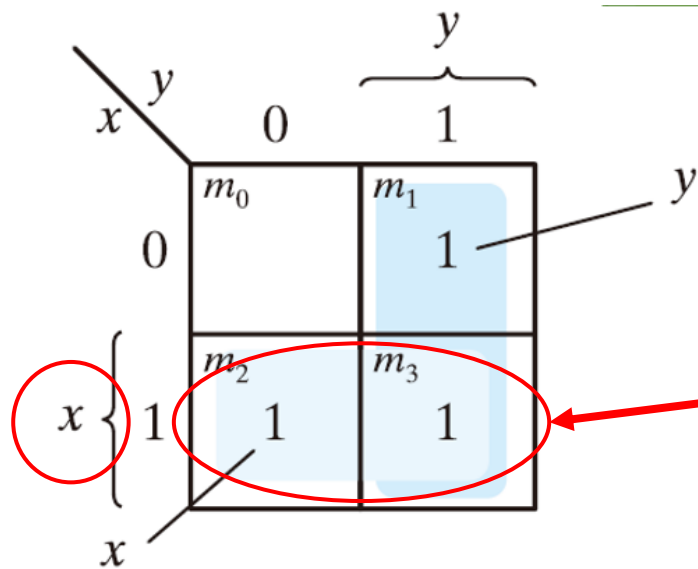


(b) $x + y$

圖 3.2 函數在卡諾圖中的表示法

二變數卡諾圖

- 在卡諾圖中，若有兩個或多個相鄰的格子，其值都是 1，我們就可以把它們合併成一個群組，而這個合併動作在布林代數上等同於消去某些變數，因此可以得到更簡單的布林表示式。



假設有兩個 minterm：

$$xy + xy'$$

兩項只有 y 不同：

因為：

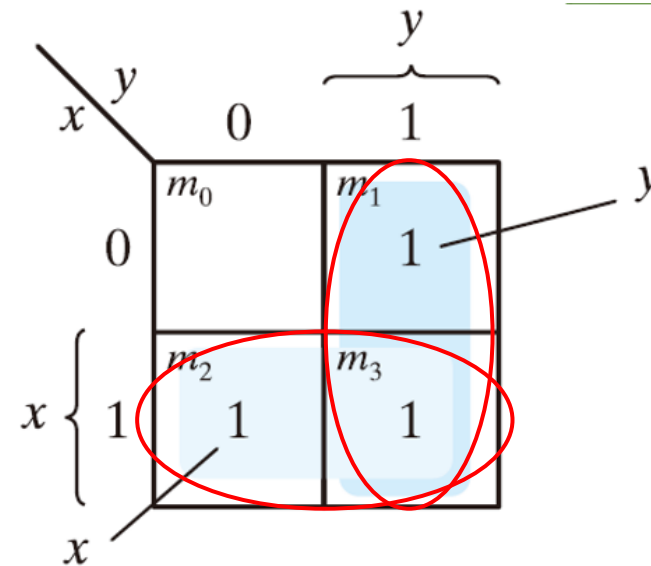
$$xy + xy' = x$$

y 被消掉了 (合併成一個群組)。

(b) $x + y$

二變數卡諾圖

- ▶ 在卡諾圖中，每一個圈出來的群組代表一個 **AND** 項，而所有群組之間用 **OR** 連接，因此最後的函數就是「積項的和 (Sum of Products)」。
- ▶ 為什麼群組之間要用 **+**
- ▶ 因為函數輸出為 1 的條件是：
- ▶ **群組1成立 或 群組2成立**
- ▶ 只要其中一個成立，輸出就為 1。
- ▶ 所以是 **OR** 關係：
- ▶ $F = y \text{ OR } x$
- ▶ 布林代數中 **OR** 用 **+** 表示：
- ▶ $F = x + y$



(b) $x + y$

三變數卡諾圖

- ▶ 因為三個變數共有八個全及項，因此圖中包含八個方格。
- ▶ 在圖 3.3 (a) 中，可以看到三變數卡諾圖的基本排列方式：
- ▶ 上排： m_0, m_1, m_3, m_2 ，下排： m_4, m_5, m_7, m_6 。
- ▶ 這個順序不是一般的二進位順序，而是 **格雷碼 (Gray Code)** 排列。
- ▶ 這樣排列的特色是相鄰的格子只會有一個變數改變。

		yz			
		00	01	11	10
x	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

		yz			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

三變數卡諾圖

- 相鄰兩方格中的兩個全及項的和可化簡為單一的AND 項，例如 m_5 與 m_7 相鄰的兩個方格的和：

$$m_5 + m_7 = xy'z + xyz = xz(y' + y) = xz$$

		yz			
		00	01	11	10
x	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

		yz			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

Diagram illustrating the simplification of $m_5 + m_7$ using a 3-variable Karnaugh map. The map shows the variables x , y , and z . The cells m_5 and m_7 are adjacent, and their sum is simplified to xz by grouping them together.

例題 3.1

► 化簡布林函數

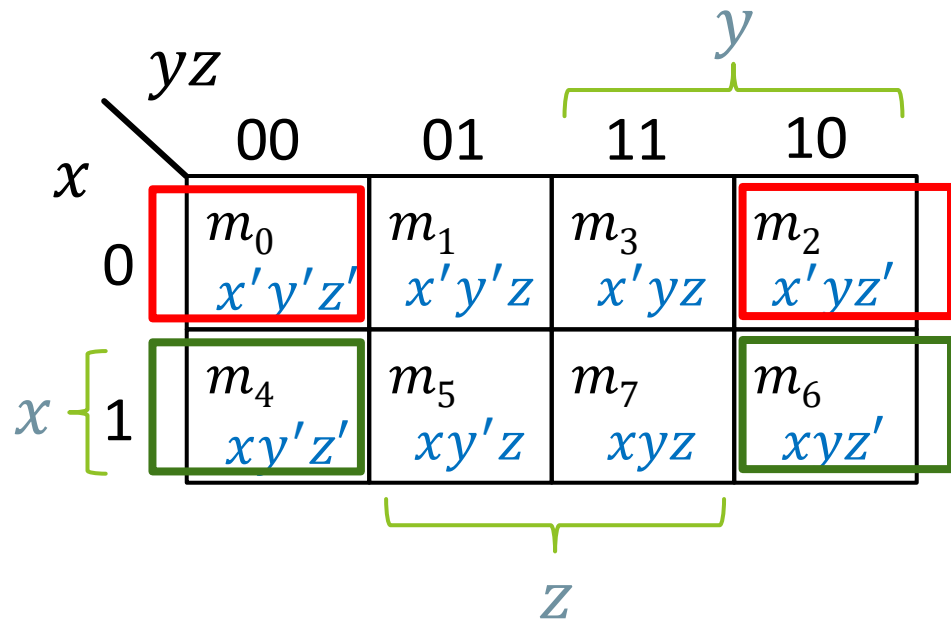
$$F(x, y, z) = \sum (2, 3, 4, 5)$$

		y			
		yz			
		00	01	11	10
x	0	m_0	m_1	m_3 1	m_2 1
	1	m_4 1	m_5 1	m_7	m_6

$$\text{Ans: } F = x'y + xy'$$

三變數卡諾圖

- 在圖中有些例子看起來兩個方格沒有接觸在一起，我們也是會將它們看成是相鄰的兩個方格。
- 圖3.3(b) 中， m_0 與 m_2 相鄰，且 m_4 與 m_6 相鄰，因為它們只相差一個變數。這個結果可以由代數證明：



例題 3.2

► 化簡布林函數

$$F(x, y, z) = \Sigma(3, 4, 6, 7)$$

		yz			
		00	01	11	10
x	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

$$\text{Ans: } F = yz + xz'$$

三變數卡諾圖

- ▶ 在三個變數的卡諾圖中，考慮任何四個相鄰的方格，任何這樣的組合表示邏輯上的四個全及項的和，並且表示式的結果只有一個變數。

$$\begin{aligned}m1 + m3 + m5 + m7 &= x'y'z + x'yz + xy'z + xyz \\&= x'z(y' + y) + xz(y' + y) \\&= x'z + xz = z\end{aligned}$$

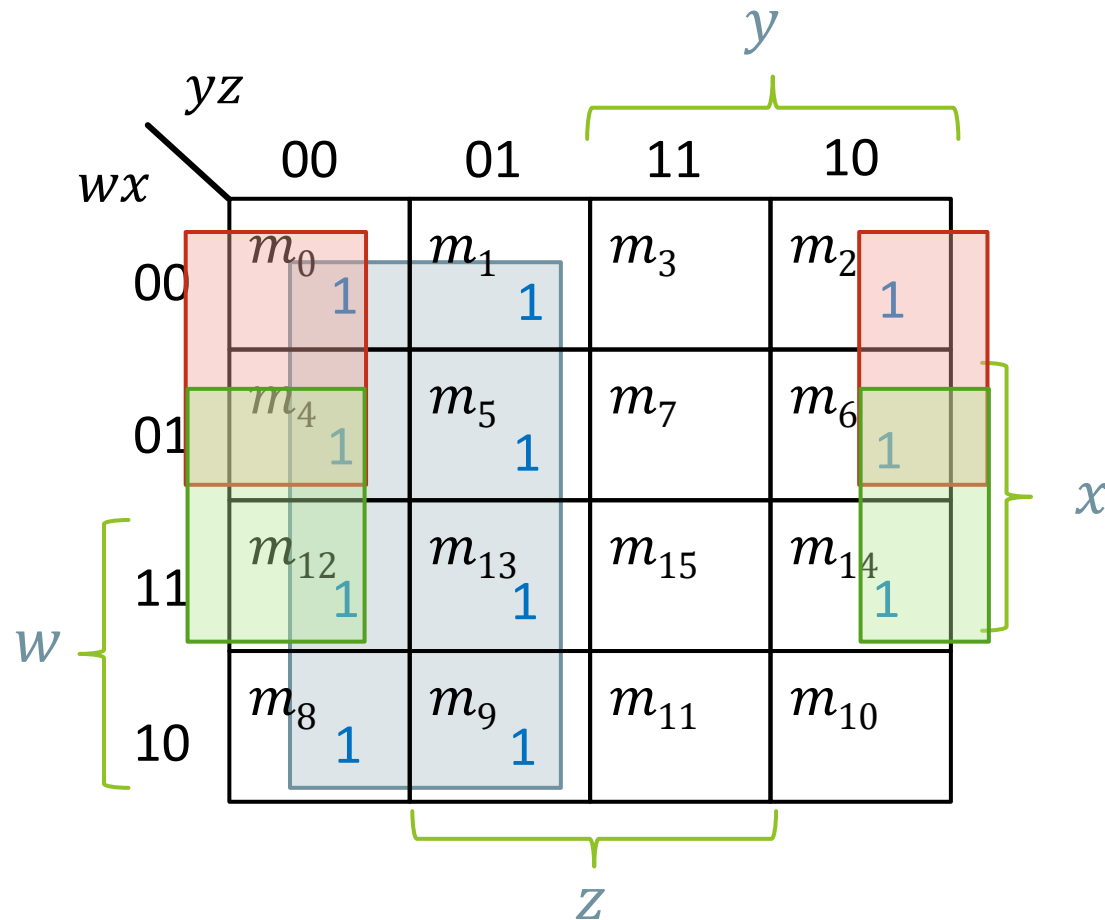
四變數卡諾圖

- ▶ 16 個全及項
- ▶ 由 2、4、8 和 16 個相鄰正方形組合而成
 - ▶ 每一個方格代表一個全及項，每一項有四個字元。
 - ▶ 兩個相鄰方格代表三個字元的項。
 - ▶ 四個相鄰方格代表二個字元的項。
 - ▶ 八個相鄰方格代表一個字元的項。
 - ▶ 十六個相鄰方格得出值永遠等於 1 的函數。

		y			
		yz			
		00	01	11	10
wx	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$
		z			
		x			

例題 3-5 化簡布林函數

$$F(w, x, y, z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$$



$$F(w, x, y, z) = y' + w'z' + xz'$$

質含項

- ▶ 所謂**質含項(prime implicant)**就是在卡諾圖中可以合併的最大可能相鄰的方格所得到的積項。換句話說：我們要盡可能把群組圈到最大，不能再擴大時，這個群組對應的項就叫 **Prime Implicant**。
- ▶ 如果 **某一個 minterm (某個 1)** 只被一個質含項覆蓋，那麼這個質含項就叫做**基本質含項 (Essential Prime Implicant)**。意思是：這個群組是必須存在的，不能被其他群組取代。如果拿掉它，就會有某個 1 沒有被覆蓋。

質含項

- ▶ 由卡諾圖去找出簡化的表示式的步驟時，首先必須決定所有的**基本質含項**。
- ▶ 簡化的表示式就是所有基本質含項之邏輯的和，再加上其他包含任何沒有被基本質含項所包含而剩下的全及項之質含項。
- ▶ 合併方格的方式不只一種，並且每一種組合可能都會產生一個同樣簡化的表示式。

練習題 3.7

- ▶ 求出布林函數 $F(w, x, y, z) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 14, 15)$ 的質含項。

Ans: $x'y', xz, xy, w'x, w'z', yz'$

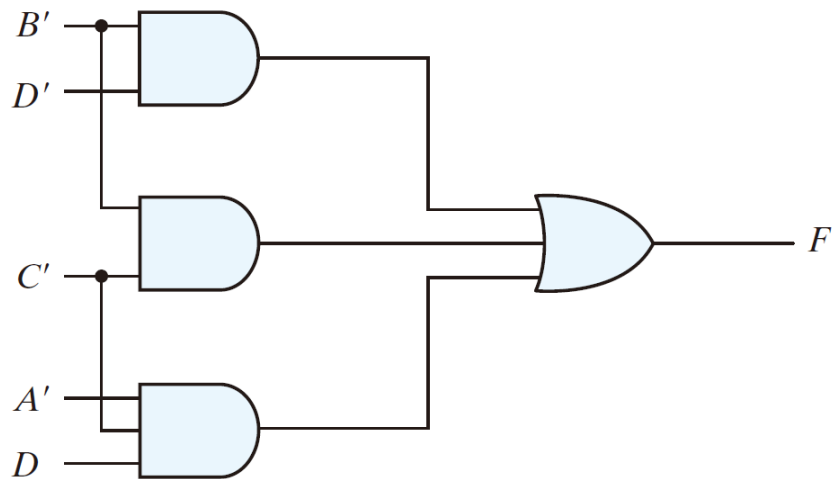
和項積的化簡

- ▶ 在利用和項積 (POS) 形式進行布林函數化簡時，主要依據布林代數的基本性質來進行分析。在卡諾圖中，將標示為「1」的方格視為函數的全及項；相對地，未被函數涵蓋的那些標準全及項，則構成該函數的補函數。
- ▶ 因此，可以理解為：卡諾圖中所有未標示「1」的方格，實際上對應的就是函數的補函數 F' 。若將這些方格以「0」表示，並依據卡諾圖中相鄰方格的合併原則，將這些「0」群組化，便可得到補函數 F' 的簡化結果。
- ▶ 最後，對 F' 再取補數，即可還原原始函數 F 。根據一般化的迪摩根定理 (De Morgan's Theorem)，這樣得到的結果會自然呈現為和項積 (POS) 形式。

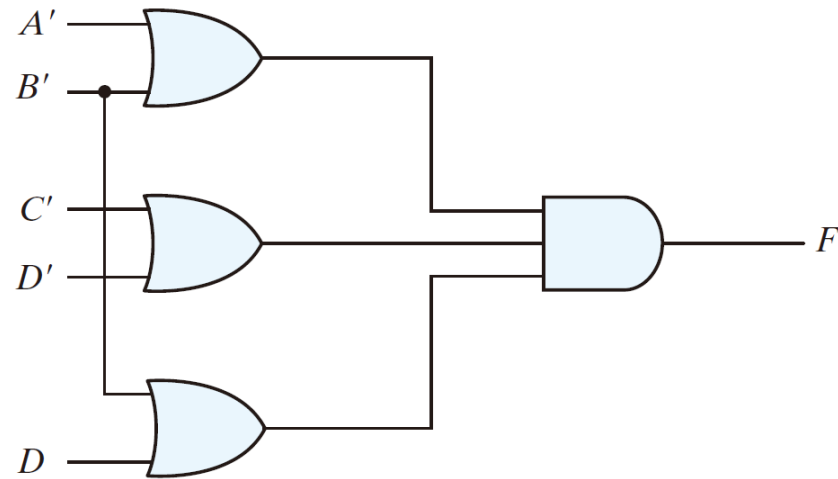
例題 3.7

- ▶ 將下列布林函數化簡為：(a) 積項和的形式；(b) 和項積的形式：
- ▶ $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$
- ▶ 此函數積項和的簡化形式：
(a) $F = B'D' + B'C' + A'C'D$
- ▶ 補函數的簡化式：
 $F' = AB + CD + BD'$
- ▶ 再應用迪摩根定理，可得函數和項積的簡化形式：
(b) $F = (A' + B')(C' + D')(B' + D)$

例題3.7函數的閘電路實現



(a) $F = B'D' + B'C' + A'C'D$



(b) $F = (A' + B')(C' + D')(B' + D)$

FIGURE 3.13

Gate implementations of the function of Example 3.7

不理會條件

- ▶ 在某些情況下，對於某些輸入組合，函數的輸出其實是**未定義的**，或者說 **不重要的**。當函數對某些輸入組合的輸出沒有被指定時，我們稱這個函數為：**不完全指定函數 (incompletely specified functions)**。通常這些未指定輸出的 minterm 就稱為：**不理會條件 (Don't-care conditions)**。
- ▶ 不理會條件可以使用在卡諾圖上，以提供布林表示式更進一步的化簡
 - ▶ 一個不理會全及項就是一個邏輯值未指定的變數組合
 - ▶ 不理會條件在卡諾圖中最初被標示為 “X”
 - ▶ 不理會條件函數 F 可以假設為 “0” 或是 “1” 的值

不理會條件

- ▶ 為什麼會出現不理會條件呢？
- ▶ 在實際系統中，有些輸入組合**根本不會出現**。
例如：
- ▶ BCD 碼只會有 **0000 到 1001**
- ▶ **1010 到 1111** 根本不會用到
- ▶ 因此對於這些輸入組合，我們不需要指定輸出是 **0 還是 1**。

練習題3.9

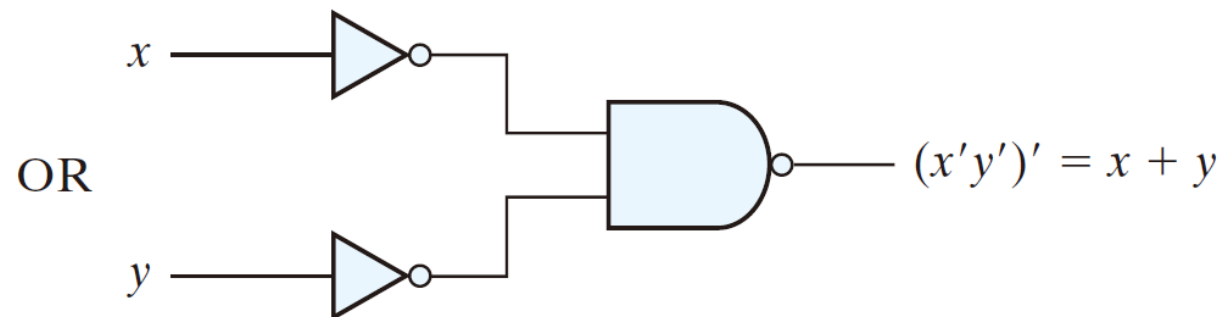
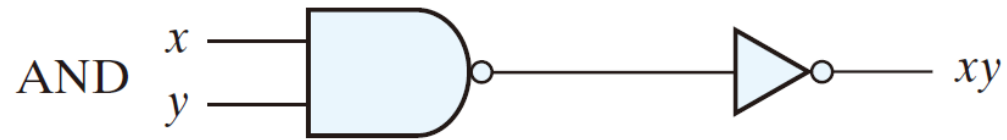
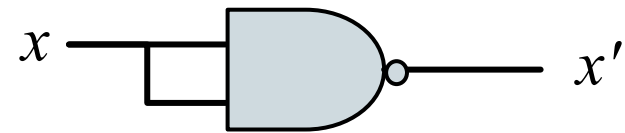
- ▶ 以不理會函數 $d(w, x, y, z) = \Sigma(0, 8, 13)$ 化簡布林函數 $F(w, x, y, z) = \Sigma(4, 5, 6, 7, 12)$

$$\text{Ans: } F = w'x + xy'$$

NAND 及 NOR 閘電路

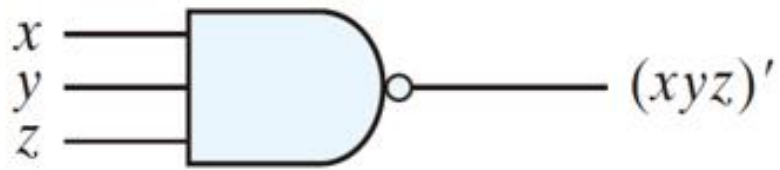
NAND 電路

- ▶ NAND 電路 NAND 閘又稱為萬用閘(universal gate)
 - ▶ 任何布林函數均可以利用 NAND 閘實現

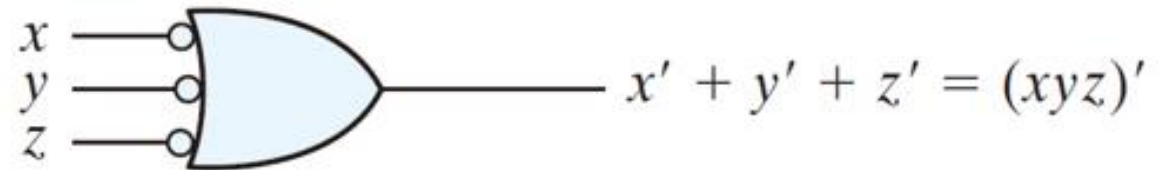


NAND 電路

兩個NAND 閘等效的圖示符號



(a) AND-invert



(b) Invert-OR

例題3.9

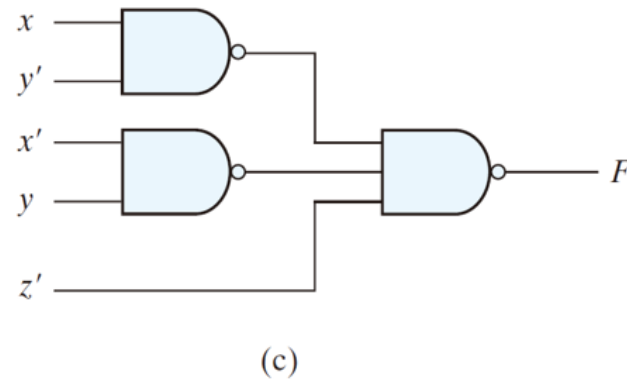
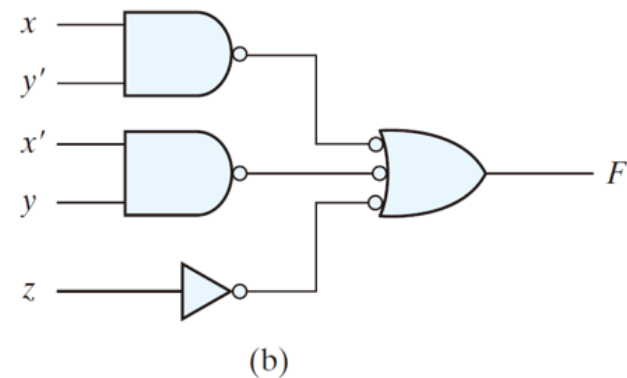
- 利用NAND閘完成下列布林函數: $F(x, y, z) = \Sigma(1,2,3,4,5,7)$

		yz			
		00	01	11	10
x	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

Groupings in the Karnaugh map:
- A green group of three cells: m_2, m_3, m_7 (value 1).
- A yellow group of four cells: m_1, m_3, m_5, m_7 (value 1).
- An orange group of two cells: m_4, m_5 (value 1).

$$F = xy' + x'y + z$$

$$\begin{aligned} F &= (xy' + x'y + z)'' \\ &= [(xy')'(x'y)'z']' \end{aligned}$$



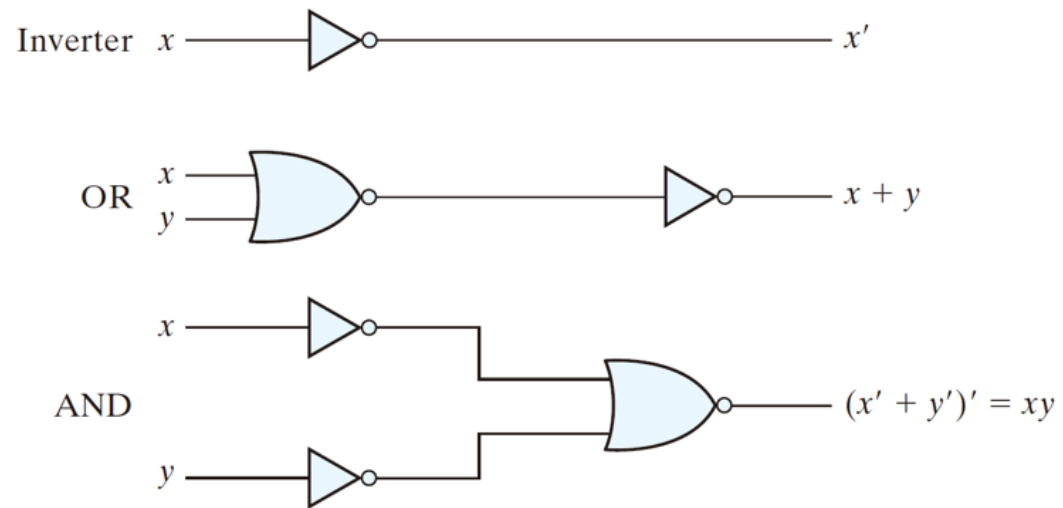
NAND 電路

► 從布林函數推導出NAND 邏輯圖的步驟如下：

1. 將函數簡化成積項和的形式。
2. 將表示式中至少含有兩個字元的每一個積項用一個NAND 閘來表示。每個NAND 閘的輸入為每一項中的變數。此步驟構成邏輯電路圖的第一階邏輯閘。
3. 在第二階的部分使用單一個AND-反相或反相-OR 閘的圖示符號來表示，而其輸入來自第一階閘的輸出。
4. 若函數中有任一項只含一個文字(變數)，則在第一階需要一個反相器。然而，若將單一個字元取補數，則可以直接將它連接到第二階NAND 閘的一個輸入。

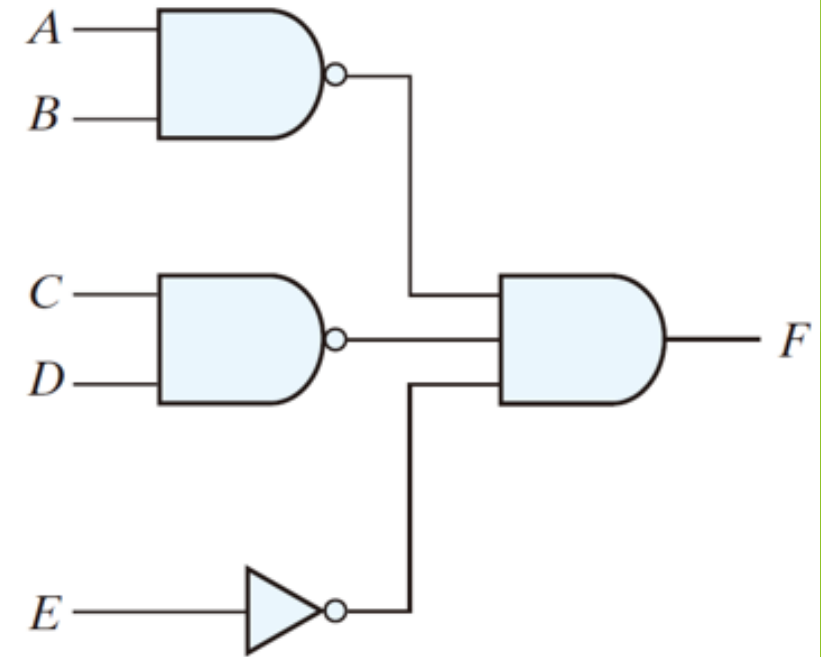
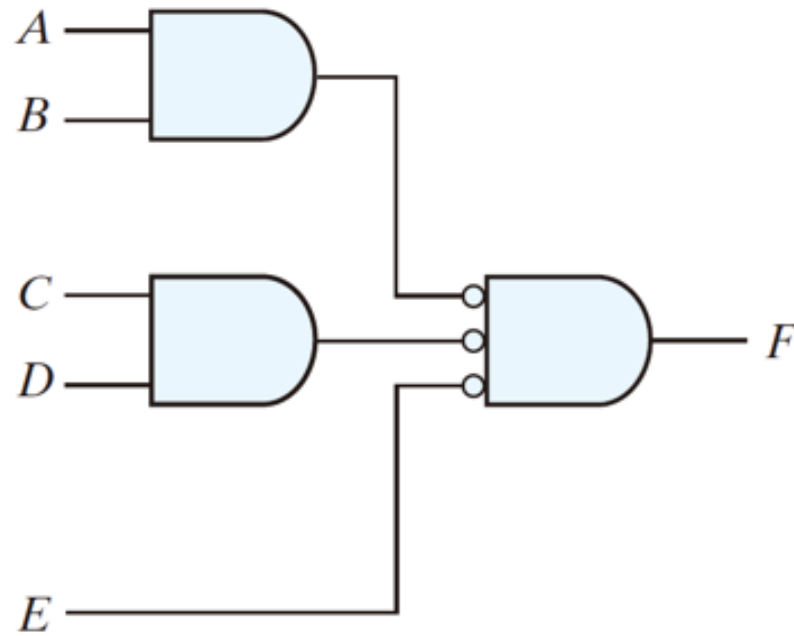
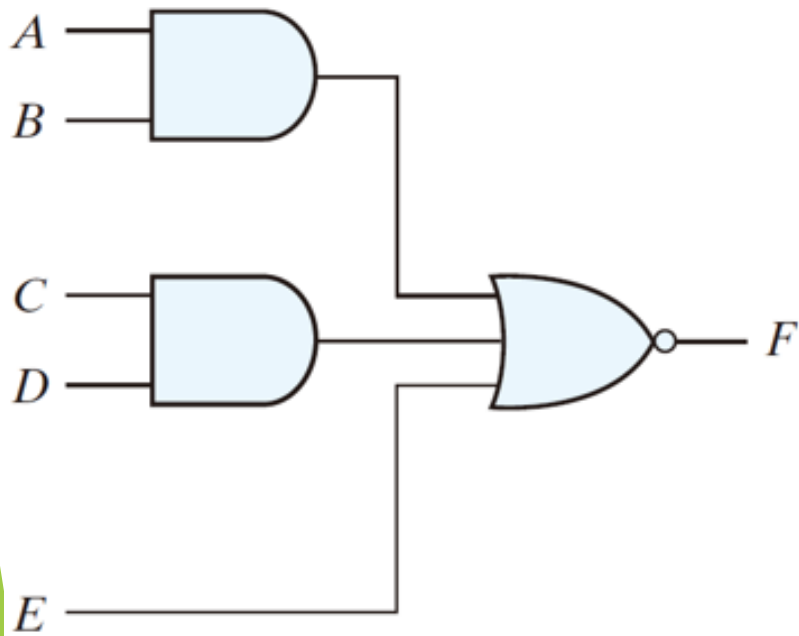
NOR閘電路的實現

- ▶ NOR 運算為NAND 運算的對偶運算。
- ▶ NOR 閘是另外一個萬用閘，可以用來實現任何的布林函數。



其他二-階層電路

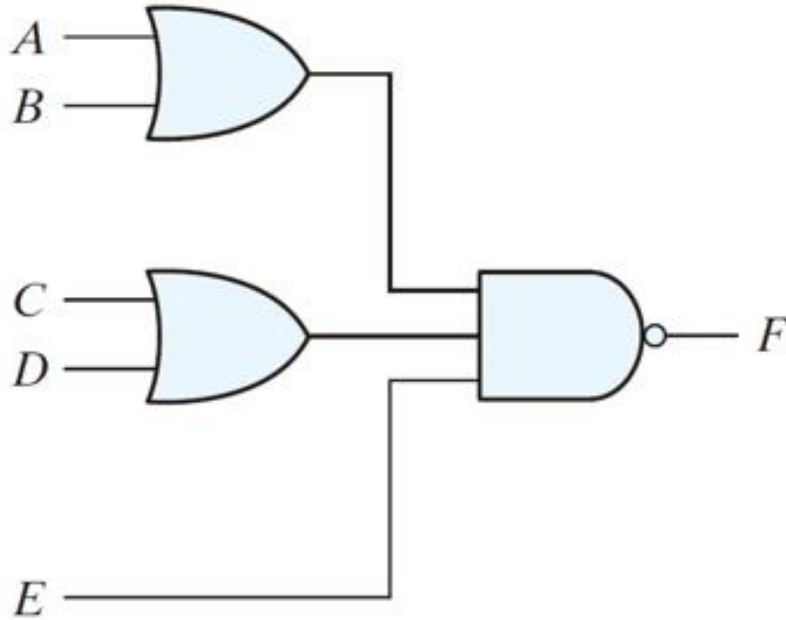
AND-OR-Invert Implementation



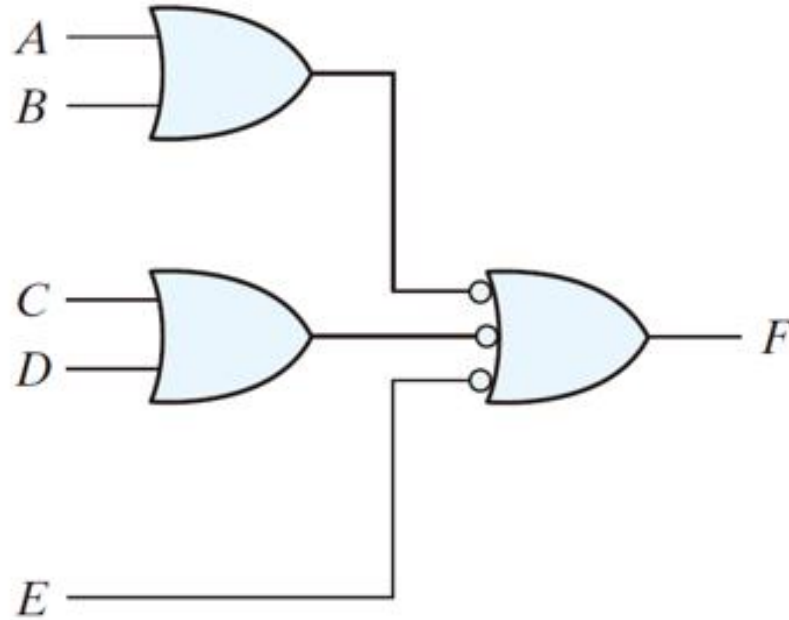
AND-OR-INVERT circuit, $F = (AB + CD + E)'$

其他二-階層電路

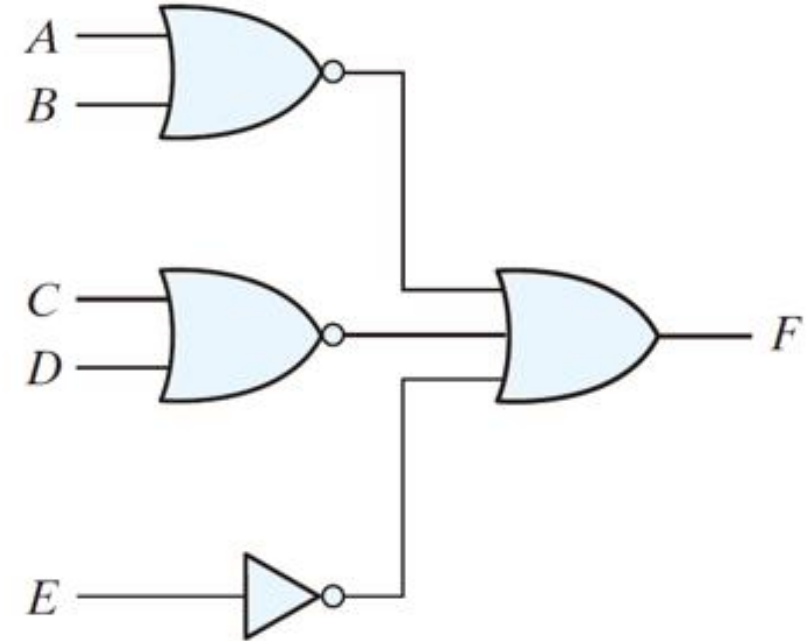
OR-AND-Invert Implementation



(a) OR-NAND



(b) OR-NAND



(c) NOR-OR

AND-OR-INVERT circuit, $F = [(A + B)(C + D)E]'$

互斥-OR 函數 (Exclusive-OR Function)

$$x \oplus y = xy' + x'y$$

- ▶ 互斥-OR (XOR) 以符號 $\oplus$ 來表示

$$(x \oplus y)' = xy + x'y'$$

- ▶ 互斥-NOR，也就是全等(equivalence)

$$x \oplus 0 = x$$

- ▶ 恆等式：

$$x \oplus 1 = x'$$

$$x \oplus x = 0$$

$$x \oplus x' = 1$$

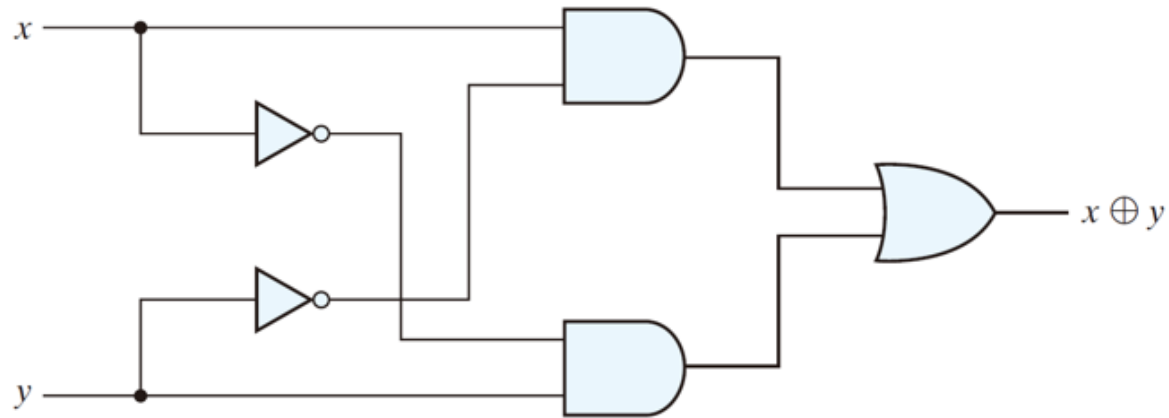
$$x \oplus y' = x' \oplus y = (x \oplus y)'$$

- ▶ 交換性與結合性

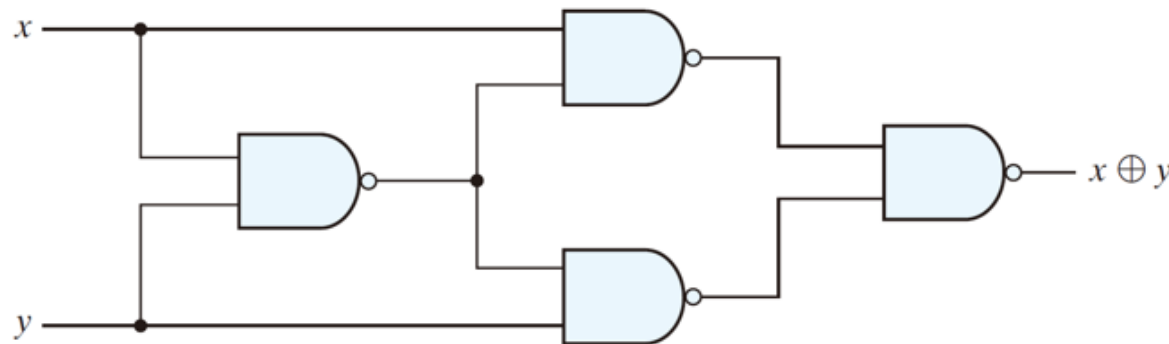
$$A \oplus B = B \oplus A$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

$$(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$$



(a) Exclusive-OR with AND-OR-NOT gates



(b) Exclusive-OR with NAND gates

$$\begin{aligned} xy' &= xy' + 0 \\ &= xy' + xx' \\ &= x(y' + x') \\ &= x(xy)' \end{aligned}$$

$$\begin{aligned} x'y &= x'y + 0 \\ &= x'y + yy' \\ &= y(x' + y') \\ &= y(xy)' \end{aligned}$$

**Exclusive-OR
implementations**

奇函數

►
$$\begin{aligned} A \oplus B \oplus C &= (AB' + A'B)C' + (AB' + A'B)'C = (AB' + A'B)C' + (AB + A'B')C \\ &= AB'C' + A'BC' + ABC + A'B'C \\ &= \Sigma(1,2,4,7) \end{aligned}$$

► 要奇數個變數等於1，函數即等於1

		BC			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

Diagram (a) shows the truth table for the odd function $F = A \oplus B \oplus C$. The variables A, B, and C are indicated by brackets. The function value is 1 for minterms m_1, m_2, m_4, m_7 .

(a) Odd function $F = A \oplus B \oplus C$

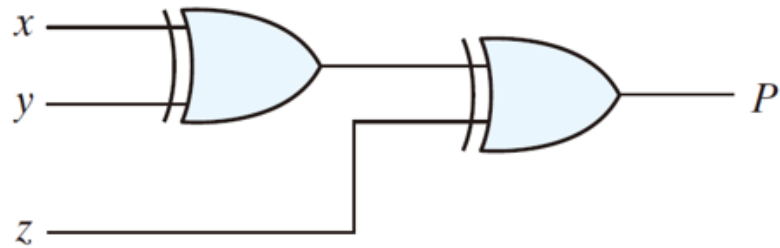
		BC			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

Diagram (b) shows the truth table for the even function $F = (A \oplus B \oplus C)'$. The variables A, B, and C are indicated by brackets. The function value is 1 for minterms m_0, m_3, m_5, m_6 .

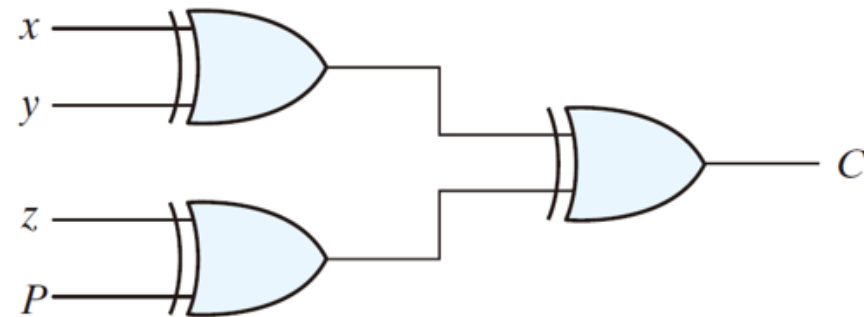
(b) Even function $F = (A \oplus B \oplus C)'$

奇函數

- ▶ 利用二-輸入互斥-OR 閘來完成的三-輸入奇函數的電路圖如圖3.32(a) 所示。
- ▶ 一個奇函數的補數可以利用將輸出以互斥-NOR 閘來取代而得，如圖3.32(b)所示。



(a) 3-bit even parity generator



(b) 4-bit even parity checker

同位產生與檢查

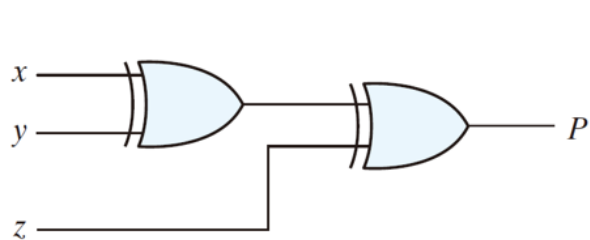
- ▶ 在傳送器中產生同位位元的電路稱為**同位產生器(parity generator)**

- ▶ $P = x \oplus y \oplus z$

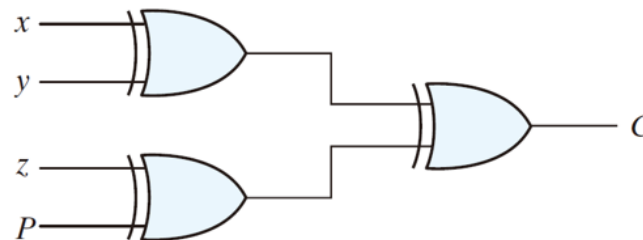
- ▶ 在接收器中檢查同位的電路稱為**同位檢查器(parity checker)**

- ▶ $C = x \oplus y \oplus z \oplus P$

- ▶ $C = 1$: 奇數個資料位元錯誤發生
- ▶ $C = 0$: 資料正確或偶數個資料位元錯誤發生



(a) 3-bit even parity generator



(b) 4-bit even parity checker